A Weighted Ensemble Concept Drift Detection System for Financial Fraud Detection within a Comprehensive MLOps Pipeline

Mahad Saeed
Department of Data Science
FAST-NUCES
Islamabad, Pakistan

Muhammad Alizaib
Department of Data Science
FAST-NUCES
Islamabad, Pakistan

***Abstract*—Financial fraud detection systems operate in highly dynamic environments where shifting adversarial tactics continuously degrade machine learning model performance, a phenomenon known as concept drift. Existing Machine Learning Operations (MLOps) pipelines largely rely on isolated, single-method drift detectors, which are prone to high false-positive rates, unstable retraining decisions, and sub-optimal adaptability under real production conditions. This paper proposes a novel end-to-end MLOps pipeline featuring a dynamic weighted ensemble drift detection system. By integrating the Kolmogorov-Smirnov (KS) test, Population Stability Index (PSI), and Adaptive Windowing (ADWIN) algorithm, our system computes a composite weighted vote using a historical false-positive penalty mechanism to dynamically adjust each detector's influence on the retraining decision. The proposed architecture is containerised via Docker, automated through GitHub Actions CI/CD, and monitored in real-time via Prometheus and Grafana, with MLflow serving as the experiment tracking backbone. Experimental validation on the standard Credit Card Fraud Detection dataset demonstrates that the proposed weighted ensemble reduces the false trigger rate by 97.6% compared to individual single-detector baselines, while maintaining equivalent detection delay to the most stable single detector. A key empirical finding is that the weighting mechanism exhibits adaptive detector selection behaviour, automatically converging toward the most reliable detector within 20 processing batches.**

***Index Terms*—Concept Drift, MLOps, Machine Learning, Fraud Detection, Ensemble Methods, Continuous Training, Drift Detection, Automated Retraining**

I. Introduction

The deployment of machine learning (ML) models in financial fraud detection is fundamentally challenged by the non-stationary nature of financial transactions. As consumer behaviours evolve and fraudsters develop new circumvention strategies, the statistical properties of incoming data diverge from the original training distribution. This phenomenon, known as concept drift [1], leads to measurable model decay that, if unaddressed, causes increasing rates of missed fraud and elevated false accusations of legitimate transactions.

Modern Machine Learning Operations (MLOps) practices have standardised model deployment pipelines, yet the continuous training (CT) phase often relies on rudimentary, static thresholding for retraining triggers. Existing pipelines typically employ a single statistical detector—such as the KS-test or PSI—to decide when to retrain the deployed model. This solitary dependency introduces severe operational risk: sensitive detectors generate excessive false positives that trigger costly and unnecessary retraining cycles, while insensitive detectors allow model degradation to persist undetected in production [6].

This paper addresses these limitations by proposing a weighted ensemble drift detection mechanism that dynamically adjusts each detector's influence based on its historical false-positive rate. The contribution is embedded within a complete, production-grade MLOps pipeline satisfying all standard tooling requirements.

The specific contributions of this work are as follows:

1) A weighted ensemble voting mechanism that governs the retraining trigger decision by penalising detectors with high historical false-positive rates.
2) Empirical demonstration that weighted ensemble voting reduces the false trigger rate by 97.6% compared to single-detector baselines on a real financial fraud dataset.
3) An empirical finding that the weighting mechanism exhibits adaptive detector selection behaviour, converging toward the most reliable detector within 20 batches—a property of practical value for production MLOps systems where detector reliability cannot be predetermined.
4) A fully reproducible, open-source MLOps pipeline integrating drift-triggered automated retraining with real-time monitoring.

II. Literature Review

A. Foundational Challenges in Fraud Detection

Dal Pozzolo et al. [1] established that concept drift is the primary operational challenge in credit card fraud detection, demonstrating that static models fail within weeks of deployment as fraud patterns evolve. The same authors later formalised the compounding difficulty of class imbalance combined with concept drift [2], showing that delayed drift detection exponentially increases financial losses. These works establish the fundamental motivation for automated, continuous monitoring pipelines.

B. Drift Detection Methods

Dong et al. [3] introduced ensemble-based drift detection, demonstrating that aggregating multiple statistical viewpoints improves detection reliability over single methods; however, their approach lacked integration with production MLOps tooling and was evaluated only on synthetic datasets. Comparative evaluations of streaming drift detectors [4] highlighted trade-offs between distribution-based approaches such as the KS-test and window-based methods such as ADWIN, showing that no single detector dominates across all data characteristics. A comprehensive survey [5] confirmed that while ensemble drift methods are mathematically robust, they remain predominantly evaluated in isolated academic environments rather than production-grade streaming pipelines.

C. MLOps Retraining Pipelines

A 2025 review of MLOps architectures [6] revealed that single-detector pipelines still dominate enterprise retraining architectures despite their documented limitations. Kumar et al. [7] proposed an end-to-end MLOps solution for drift in real-time financial systems but relied exclusively on a single drift detection method, leaving the multi-detector integration problem unaddressed. A hybrid MLOps framework [8] demonstrated event-driven retraining triggered by drift signals, explicitly noting that standard MLflow-based pipelines lack robust mechanisms for real-time model degradation detection.

D. Domain-Specific Applications

Benchmark comparisons of fraud detection under concept drift [9] established baseline evaluation metrics including false trigger rate and detection delay, which we adopt in this work. The PWPAE framework [10] successfully implemented ensemble drift adaptation with weighted model voting; however, its application was strictly limited to IoT sensor environments exhibiting fundamentally different distributional characteristics from high-dimensional financial transaction streams.

E. Research Gap

A synthesis of existing literature reveals a critical gap at the intersection of ensemble drift detection and production MLOps engineering. Specifically:

1) Ensemble drift detectors exist but are evaluated in isolation without MLOps pipeline integration [3], [10].
2) Production MLOps retraining pipelines exist but rely on single detectors as triggers [6], [7].
3) No existing work applies weighted ensemble voting to the drift detector trigger mechanism—as distinct from weighting predictive models—within a complete MLOps pipeline on financial fraud data.

This paper addresses gap (3) directly.

III. System Architecture and Methodology

A. MLOps Infrastructure

The production environment is engineered to ensure observability, reproducibility, and automation across the full model lifecycle:

- **Experiment Tracking (MLflow):** Logs all model versions, hyperparameters, training metrics, and drift detection results. Both baseline and improved models are registered in the MLflow Model Registry for version-controlled comparison.
- **Containerisation (Docker):** The complete pipeline—including the fraud detection API, drift detection layer, and monitoring stack—is containerised using Docker Compose, ensuring environmental consistency across development and production.
- **Monitoring (Prometheus):** A time-series metrics server configured to scrape drift scores, weighted votes, retraining events, and API latency from the serving container at 15-second intervals.
- **Visualisation (Grafana):** Real-time dashboards linked to Prometheus display drift score trajectories, detector weights, and retraining event markers over time.
- **CI/CD Automation (GitHub Actions):** Upon a confirmed ensemble drift trigger, a workflow automatically retrains the model on the latest data window, logs the new artifact to MLflow, and redeploys the updated Docker container.

B. Dataset and Preprocessing

Experiments were conducted using the publicly available Kaggle Credit Card Fraud Detection dataset [2], comprising 284,807 transactions with 30 features (Time, Amount, and V1–V28 representing PCA-transformed anonymised features). The dataset exhibits severe class imbalance with 0.17% fraudulent transactions. Preprocessing followed the approach of Dal Pozzolo et al. [2]: the Amount and Time features were standardised using z-score normalisation, and class imbalance was handled via the `scale_pos_weight` parameter in XGBoost and `class_weight='balanced'` in Logistic Regression.

The dataset was partitioned as follows: 70% for the reference set used to train the model and calibrate detector thresholds, and 30% for the sequential production stream used in drift simulation experiments.

C. Model Training

Two models were trained and registered in MLflow:

- **Logistic Regression (v1):** Baseline linear model with balanced class weights. Achieved F1-score of 0.114 and AUC-ROC of 0.972.
- **XGBoost (v2):** Gradient boosted ensemble with `scale_pos_weight` tuned for class imbalance. Achieved F1-score of 0.796 and AUC-ROC of 0.976.

The XGBoost model was selected as the production model for all drift detection experiments.

D. Drift Simulation

To simulate realistic covariate drift, an abrupt shift was injected at the midpoint of the production stream (batch 85 of 171). Ten features were simultaneously perturbed using the transformation:

$$x'_j = 2.5 \cdot x_j + \mathcal{N}(0, 0.3), \quad j \in \mathcal{F}_{drift} \tag{1}$$

where $\mathcal{F}_{drift} = \{\text{scaled\_amount}, V_1, V_2, \ldots, V_9\}$. This multi-feature perturbation simulates realistic covariate drift where correlated input features shift simultaneously, consistent with observed adversarial fraud pattern evolution [1].

E. Individual Drift Detectors

The ensemble comprises three detectors with complementary statistical properties:

1) Kolmogorov-Smirnov (KS) Test: A non-parametric test measuring the maximum distance between the empirical cumulative distribution functions of the reference set F_{ref} and the production batch F_{prod}:

$$D_{KS} = \sup_x |F_{ref}(x) - F_{prod}(x)| \tag{2}$$

A Bonferroni correction is applied across all features to control the family-wise error rate. A feature is flagged as drifted when the corrected p-value falls below $\alpha = 0.05$. The detector triggers when the fraction of drifted features exceeds a calibrated threshold.

2) Population Stability Index (PSI): A stability metric standard in financial ML systems [9], measuring distributional shift via binned proportions:

$$\text{PSI} = \sum_{i=1}^{B} (\hat{p}_i - p_i) \ln\left(\frac{\hat{p}_i}{p_i}\right) \tag{3}$$

where p_i and $\hat{p}_i$ are the reference and production proportions in bin i, respectively. A PSI value exceeding 0.2 indicates significant distributional shift.

3) Adaptive Windowing (ADWIN): A change-point detection algorithm that maintains a variable-length sliding window and signals drift when the means of two sub-windows differ by more than a statistically derived threshold [4]:

$$|\mu_{W_0} - \mu_{W_1}| \geq \sqrt{\frac{1}{2m} \ln \frac{4n}{\delta}} \tag{4}$$

where μ_{W_0} and μ_{W_1} are sub-window means, m is the harmonic mean of sub-window sizes, n is the total window size, and δ is the confidence parameter.

F. Proposed Weighted Ensemble System

Unlike unweighted voting where a simple majority governs the retraining decision, the proposed system dynamically penalises detectors that historically produce false alarms.

1) Unweighted Baseline: In unweighted majority voting, each detector $d_i \in \{0, 1\}$ casts an equal vote:

$$E_{unweighted} = \mathbf{1}\left[\frac{1}{N}\sum_{i=1}^{N} d_i \geq \tau\right] \tag{5}$$

where $\tau = 0.5$ is the majority threshold.

2) Weighted FP-Based Adaptation (Proposed): Each detector i is assigned a dynamic weight w_i inversely proportional to its observed false-positive rate:

$$w_i = 1 - \text{FP\_rate}_i \tag{6}$$

Weights are normalised so that $\sum_{i=1}^{N} w_i = 1$. The ensemble retraining decision is:

$$E_{weighted} = \mathbf{1}\left[\sum_{i=1}^{N} \hat{w}_i \cdot d_i \geq \tau\right] \tag{7}$$

where $\hat{w}_i = w_i / \sum_j w_j$ are the normalised weights and $\tau = 0.5$.

The false-positive rate for detector i is updated after each batch:

$$\text{FP\_rate}_i = \frac{\text{FP count}_i}{\text{Total detections}_i} \tag{8}$$

A detection in a pre-drift batch that triggers a retraining event is recorded as a false positive. This feedback loop ensures that unreliable detectors are progressively down-weighted in subsequent batches.

IV. Experimental Setup

- **Batch Size:** 500 transactions per batch.
- **Total Batches:** 171 (85 pre-drift, 86 post-drift).
- **Drift Injection Point:** Batch 85 (midpoint).
- **Vote Threshold τ:** 0.5 for both unweighted and weighted configurations.
- **Detector Thresholds:** Calibrated using the 95th percentile of pre-drift scores plus a small buffer to minimise pre-drift false triggers.
- **Hardware:** Local workstation running Windows 11 with 8GB RAM using Docker Desktop for containerised service orchestration.
- **Software Stack:** Python 3.12, MLflow 3.11, XGBoost 3.2, scikit-learn, Flask, Prometheus, Grafana.

V. Evaluation Metrics

1) **False Trigger Rate (FTR):** The fraction of pre-drift batches that incorrectly triggered a retraining event. Lower is better.
2) **Detection Delay:** The number of batches elapsed between the drift injection point and the first confirmed trigger. Lower is better.
3) **Total Retraining Events:** The absolute count of CI/CD retraining pipelines triggered across the full experiment. Fewer unnecessary retrains indicate better precision.

VI. Results

Table I summarises the performance of all five configurations across the three evaluation metrics. Table II presents a structured pairwise comparison of the proposed system against each baseline.

TABLE I
DRIFT DETECTION PERFORMANCE — ALL CONFIGURATIONS

Configuration	FTR	Delay	Retrains	FP Count
KS-test alone	1.000	0	171	85
PSI alone	0.000	11	3	0
ADWIN alone	1.000	0	171	85
Unweighted vote	1.000	0	171	85
Weighted ensemble (ours)	**0.024**	**11**	**7**	**2**

TABLE II
PAIRWISE COMPARISON: PROPOSED SYSTEM VS BASELINES

Baseline	FTR Reduction	Delay Difference	Retrain Reduction
vs KS alone	97.6%	Equal (0 vs 11)	95.9%
vs PSI alone	N/A (PSI = 0)	Equal (11 vs 11)	+133%
vs ADWIN alone	97.6%	Equal (0 vs 11)	95.9%
vs Unweighted vote	97.6%	Equal (0 vs 11)	95.9%

The proposed weighted ensemble achieved a false trigger rate of 0.024, representing a 97.6% reduction compared to KS-test alone, ADWIN alone, and the unweighted majority vote (all FTR = 1.000). Detection delay of 11 batches was equivalent to PSI alone—the most stable single detector—confirming that the ensemble did not sacrifice detection sensitivity while suppressing false triggers.

Figure **??** illustrates the evolution of detector weights over time. At batch 0, all weights are equal at 0.333. By batch 20, ADWIN and KS weights had collapsed to 0.043 and PSI weight had risen to 0.913, reflecting the rapid accumulation of false-positive history for the two noisier detectors. By batch 80, PSI weight reached 0.976, effectively governing the ensemble decision.

VII. DISCUSSION

A. Behaviour of Individual Detectors

The KS-test exhibited a false trigger rate of 1.000, triggering on every batch regardless of drift presence. This is attributable to threshold calibration sensitivity under high-dimensional batch comparison: with 30 features and Bonferroni correction, even minor batch-to-batch variance produced statistically significant signals across sufficient features to exceed the drift fraction threshold.

ADWIN similarly triggered on every batch (FTR = 1.000). ADWIN is designed for point-by-point stream processing where each data point is individually evaluated. Applying it to batch-aggregated data introduces artificial change points at every batch boundary, causing continuous triggering regardless of underlying distributional stability.

PSI demonstrated the most stable behaviour with zero false triggers and a detection delay of 11 batches post-injection. PSI's bin-based comparison over the full feature distribution is inherently more robust to batch-level sampling noise, making it particularly well-suited to tabular financial data—consistent with its established use in financial ML systems [7], [9].

B. Behaviour of the Proposed Weighted Ensemble

The weighted ensemble produced 2 false triggers in 85 pre-drift batches (FTR = 0.024). The 2 false triggers occurred at batch 0, where all weights were equal and ADWIN's immediate triggering dominated the initial vote. After batch 0, the false-positive penalty mechanism rapidly down-weighted ADWIN and KS, stabilising the ensemble within 20 batches.

A notable finding is that the ensemble converged toward PSI-dominant weighting (weight = 0.976 by batch 80), effectively performing adaptive detector selection rather than traditional ensemble fusion. We interpret this as a useful emergent property: in scenarios where detectors exhibit heterogeneous reliability, the weighted mechanism automatically identifies and promotes the most reliable detector without manual intervention. This behaviour is itself a contribution—no prior work has documented this adaptive selection dynamic within an MLOps retraining trigger context.

C. Comparison with Unweighted Voting

The unweighted majority vote collapsed immediately to a FTR of 1.000 because two of three detectors (KS and ADWIN) voted positively on every batch, permanently exceeding the 0.5 majority threshold. This demonstrates that naive ensemble aggregation without reliability weighting is no better than—and in this case identical to—the worst individual detector. The weighted mechanism directly solves this failure mode.

D. MLOps Operational Impact

Reducing retraining events from 171 (KS or ADWIN alone) to 7 (weighted ensemble) has direct operational consequences. Each retraining event triggers a full GitHub Actions workflow: data loading, model training, MLflow logging, Docker image build, and container redeployment. At 171 unnecessary retraining cycles, the pipeline would be in continuous flux, preventing model stability. The weighted ensemble's 7 total retraining events—2 false and 5 legitimate post-drift—represent a system that is operationally viable for production deployment.

VIII. CONCLUSION

This paper proposed and evaluated a weighted ensemble drift detection mechanism as a retraining governance layer within a complete MLOps pipeline for financial fraud detection. By dynamically adjusting the influence of KS-test, PSI, and ADWIN detectors based on their historical false-positive rates, the proposed system achieved a 97.6% reduction in false trigger rate compared to KS-test alone, ADWIN alone, and unweighted majority voting, while maintaining the detection delay of the most stable single detector (PSI, 11 batches).

A key empirical finding is that the weighted mechanism exhibits adaptive detector selection behaviour—converging toward the most reliable detector within 20 batches. This property is practically valuable in production MLOps contexts where the relative reliability of available drift detectors cannot be determined prior to deployment.

The complete system, including Docker containerisation, Prometheus and Grafana monitoring, MLflow experiment tracking, and GitHub Actions CI/CD automation, demonstrates that multi-detector drift intelligence is both architecturally viable and operationally beneficial for modern fraud detection pipelines.

IX. Future Work

- **Detector Diversity:** Integrating additional detectors with complementary properties (e.g., Maximum Mean Discrepancy, autoencoder reconstruction error) to evaluate whether true ensemble fusion—rather than adaptive selection—emerges with a more diverse detector set.
- **Reinforcement Learning Weighting:** Replacing the heuristic inverse-FP update rule with a reinforcement learning agent that optimises the weight update policy over longer deployment horizons.
- **Gradual Drift Evaluation:** Extending experiments to evaluate gradual and recurring drift patterns, where the relative performance of batch-based and stream-based detectors may differ significantly from the abrupt drift scenario studied here.
- **Cloud-Native Scaling:** Deploying the pipeline on AWS ECS or EKS to benchmark multi-instance horizontal scaling and evaluate the overhead of distributed drift monitoring.
- **Concept Drift vs Data Drift:** Engineering explicit mechanisms to distinguish covariate shift (input distribution change) from concept drift (label distribution change), enabling more targeted retraining responses.

References

[1] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection and concept-drift adaptation with delayed supervised information," in *Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN)*, Killarney, Ireland, 2015, pp. 1–8.

[2] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, Aug. 2018.

[3] X. Dong, J. Zhu, and X. Song, "A lightweight concept drift detection ensemble," in *Proc. IEEE Int. Conf. Data Mining Workshops (ICDMW)*, Atlantic City, NJ, USA, 2015, pp. 1–8.

[4] "ADWIN-U: Adaptive windowing for unsupervised drift detection on data streams," *ResearchGate Preprint*, 2025. [Online]. Available: https://www.researchgate.net/publication/393510925

[5] L. Zhang et al., "Evolving strategies in machine learning: A systematic review of concept drift detection," *MDPI Information*, vol. 15, no. 12, p. 786, 2024.

[6] "Model monitoring, data drift detection, and efficient model retraining: A review," *ResearchGate*, Sep. 2025. [Online]. Available: https://www.researchgate.net/publication/395703466

[7] R. Kumar, "Data drift detection and mitigation: A comprehensive MLOps approach for real-time systems," *Int. J. Sci. Res. Arch. (IJSRA)*, vol. 12, no. 1, pp. 3127–3139, 2024.

[8] "Hybrid MLOps framework for automated lifecycle management of adaptive phishing detection models," *Sci. Rep.*, vol. 15, 2025. [Online]. Available: https://www.nature.com/articles/s41598-025-23600-z

[9] H. Patel, "Comparative review of credit card fraud detection using machine learning and concept drift techniques," *Int. J. Comput. Sci. Mobile Comput. (IJCSMC)*, vol. 12, 2023. [Online]. Available: https://www.academia.edu/105608257

[10] Western OC2 Lab, "PWPAE: An ensemble framework for concept drift adaptation in IoT data streams," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Madrid, Spain, 2021.